SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 Assessment Tool 2 — Architecture Design Assignment

Course Code: CSA1012 | **Course Title**: Software Engineering

Student Name: Y PRAVEEN RAJ (**192372306**)

CO2: Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills

Problem Statement

Formal Verification Framework for E-Transaction Protocols Using Product Automata and Reachability Analysis for Fraud Detection.

The system takes a formal specification of an e-transaction protocol, models it as a finite-state automaton, composes it with known fraud-pattern automata (replay attacks, double spending, identity forgery, and race conditions), and uses reachability analysis (CTL model checking) to determine whether a fraudulent state can be reached. Results, automata diagrams, and fraud-coverage summaries are published on a project website.

1. Analysis of System Requirements

1.1 System Objectives

- Formally verify e-transaction protocols against a defined catalogue of fraud patterns.
- Provide clear, visual evidence (automata diagrams, CTL results) of which fraud patterns a protocol is provably safe against.
- Make the verification process repeatable, so a protocol can be re-checked whenever it changes.

1.2 Functional Requirements

Requirement	Description
Protocol ingestion	Accept a formal specification of an e-transaction protocol (states, transitions, messages exchanged between parties).
Automata construction	Convert the protocol specification into a finite-state automaton representing all legal protocol runs.
Fraud-model composition	Compose the protocol automaton with attacker/fraud-pattern automata (replay, double-spend, identity forgery, race condition) to build a product automaton.
Reachability analysis	Explore the product automaton's state space and apply CTL model checking to determine whether a fraudulent state is reachable from the initial state.
Result reporting	Generate a human-readable verification report: which fraud patterns were checked, which were found reachable/unreachable, and a counter-example trace when reachable.
Visualization	Render interactive automata and reachability diagrams on the project website for a chosen protocol/fraud-pattern combination.

1.3 Non-Functional Requirements and Quality Attributes

Quality Attribute	How it Influences the Architecture
Scalability	State-space exploration is computationally expensive and grows with protocol complexity, so the verification engine must be separable from the UI and scalable independently of it.
Performance	Reachability analysis on larger automata should not block the API or the UI thread; long-running checks need asynchronous processing.
Maintainability	New fraud patterns and protocol types are added over time, so parsing, automata construction, and the checking algorithm must be cleanly separated, replaceable modules.
Reliability	A verification result is only useful if it is deterministic and reproducible for the same protocol/fraud-pattern inputs.
Availability	The project/results website should remain viewable even if the verification engine is temporarily busy or offline.
Security	Protocol specifications and verification results should not be tampered with in transit or storage, since the output is meant to support fraud-detection claims.

2. Selection of Software Architecture

2.1 Selected Architectural Style

Hybrid Architecture: a Layered Architecture for the overall system, with the verification engine internally organized as a set of loosely-coupled, Service-Oriented components (parser, automata constructor, product-automata generator, CTL checker, report generator) reached through an API Gateway and a job queue.

2.2 Justification

- **Separation of concerns:** the strict layering (presentation → API → application/verification → data) keeps the website, the request handling, and the formal-methods logic independent, which matches how differently each layer changes over the project's lifetime.
- **Computationally heavy core:** reachability analysis over a product automaton can be expensive; treating the verification engine as a set of services behind a job queue lets that work scale or be optimized without redesigning the UI.
- **Extensibility:** new fraud-pattern automata or new protocol formats are additions to a single component (fraud pattern library or parser) rather than changes that ripple across the system, since each pipeline stage is a distinct, replaceable service.
- **Fit for project scale:** a full microservices deployment would add operational overhead (service discovery, distributed transactions) this project does not need; a layered structure with internally service-oriented engine components gives most of the scalability and maintainability benefit at much lower complexity.

2.3 Trade-offs Considered

- Pure layered architecture alone would be simpler to build but would tightly couple the model-checking algorithm to the API layer, making it harder to scale or replace independently — rejected in favour of the hybrid split.
- Full microservices (each engine stage as an independently deployed service with its own database) would maximize scalability but adds deployment and network complexity not justified for a single-team

academic/portfolio project — the hybrid approach keeps that option open for the future without paying the cost now.

- A monolithic script (no API layer at all) was rejected because it would prevent the project website from showing live, on-demand verification results.

3. Software Architecture Diagram

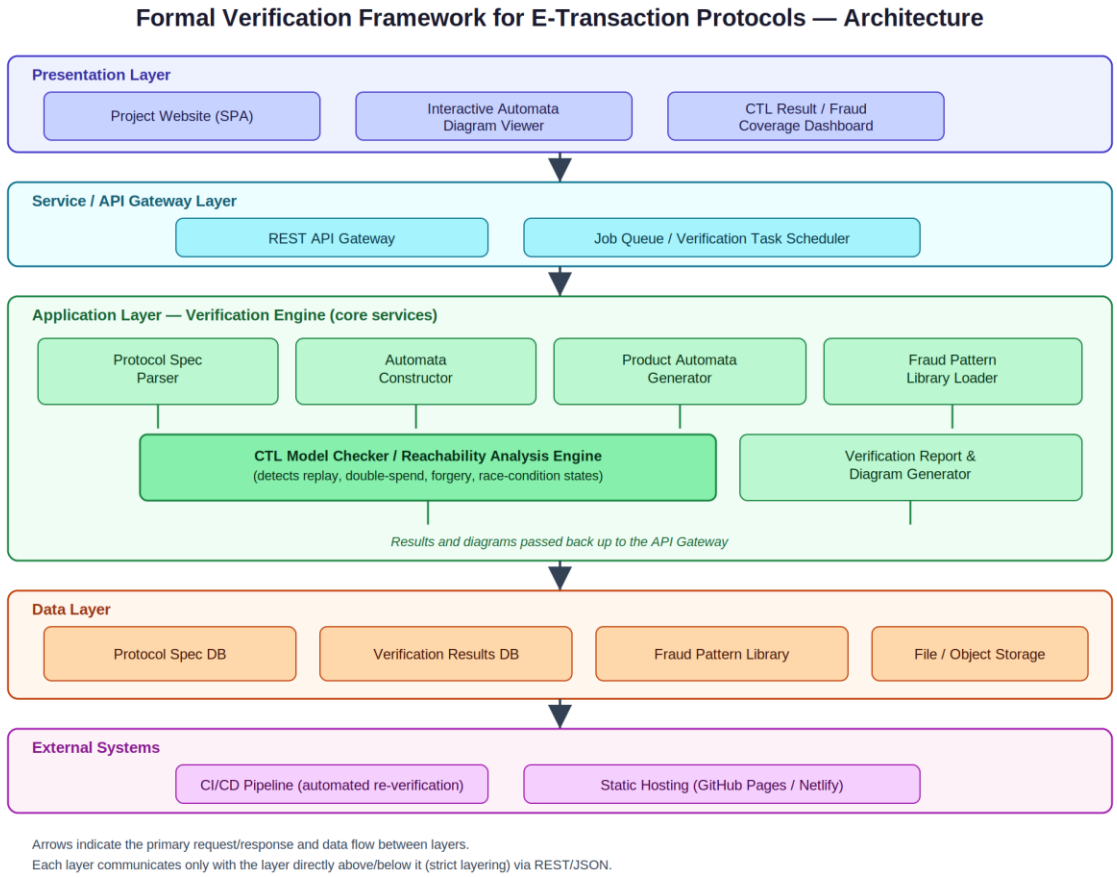

Figure 1: Layered architecture with the verification engine's internal services shown as a service-oriented cluster within the Application Layer.

4. Components and Their Interactions

Component	Responsibility
Presentation Layer (Project Website)	Dark-themed single-page site that lets a user pick a protocol/fraud pattern, view interactive automata diagrams, and read CTL verification results and fraud-coverage summaries.
REST API Gateway	Single entry point for the UI; validates requests, routes them to the verification engine, and returns results/diagrams as JSON.
Job Queue / Task Scheduler	Queues verification requests so long-running reachability checks run asynchronously without blocking the API or the UI.
Protocol Spec Parser	Reads the submitted protocol description and converts it into an internal automaton representation.
Automata Constructor	Builds the formal finite-state automaton for the protocol from the parsed specification.
Product Automata Generator	Composes the protocol automaton with a selected fraud-pattern automaton to produce the product automaton that is actually checked.
Fraud Pattern Library Loader	Supplies the catalogue of known fraud automata (replay attack, double spending, identity forgery, race condition) used during composition.
CTL Model Checker / Reachability Analysis Engine	Core verification component; explores the product automaton's reachable state space and evaluates CTL properties to decide if a fraud state is reachable.
Verification Report & Diagram Generator	Converts raw model-checking output into the JSON/SVG payload the website renders (diagrams, CTL results, coverage table, counter-example trace).
Data Layer (Protocol Spec DB, Results DB, Fraud Pattern Library, File Storage)	Persists protocol specifications, past verification runs, the fraud-pattern catalogue, and generated diagram/report files.
CI/CD Pipeline	Re-runs verification automatically whenever a protocol specification or the fraud-pattern library changes.
Static Hosting (GitHub Pages / Netlify)	Serves the public-facing project website that presents the framework and its results.

4.1 Overall Workflow

- The user selects a protocol and one or more fraud patterns to check on the Project Website.
- The browser sends a verification request to the REST API Gateway, which validates it and places a job on the Job Queue.
- The Protocol Spec Parser converts the stored specification into an automaton; the Automata Constructor finalizes the protocol automaton.
- The Product Automata Generator composes the protocol automaton with the requested fraud-pattern automaton(s) drawn from the Fraud Pattern Library.
- The CTL Model Checker performs reachability analysis on the product automaton to decide whether each fraud state is reachable, producing a counter-example trace when it is.

- The Verification Report & Diagram Generator turns this output into a report and diagram payload, which is persisted in the Data Layer and returned through the API Gateway.
- The website renders the interactive automata diagram and the CTL result / fraud-coverage dashboard from that payload.

5. Discussion of Quality Attributes

Quality Attribute	How the Architecture Addresses It
Scalability	The verification engine sits behind the API Gateway as an independently deployable service reached through a job queue, so heavier protocols or a backlog of checks can be scaled (e.g. more worker instances) without touching the UI.
Security	All communication between the browser, gateway, and engine goes through a single validated REST interface; protocol specs and results are stored server-side rather than trusted from client input, and the public site only ever displays already-verified, read-only results.
Performance	Parsing, automata construction, product-automata generation, and model checking are split into small, focused stages behind a job queue, so expensive reachability analysis runs asynchronously and does not block API responses or page loads.
Reliability	Automata construction and CTL checking are deterministic, stateless functions over stored specifications, so re-running a verification job for the same protocol/fraud pattern always reproduces the same result.
Maintainability	Each layer (presentation, API, verification engine, data) and each engine component (parser, constructor, checker, report generator) is a separate, independently replaceable module, so adding a new fraud pattern or protocol format touches only one component.
Availability	Separating the static website from the verification engine means the results/diagrams already generated stay viewable even if the engine is temporarily down for maintenance or busy processing a queue.

6. Justification of Architectural Decisions

6.1 Rationale

The architecture was chosen to mirror the two very different kinds of work the system does: presenting results to a human reader, and running formal, computationally heavy verification. Keeping those concerns in separate layers, connected only through a well-defined API, means each can be built, tested, and reasoned about independently — important for a formal-verification tool where the correctness of the checking logic must be trusted on its own.

6.2 Trade-offs

The main trade-off is added indirection: a request now crosses an API boundary and a job queue instead of calling the checker directly, which is unnecessary overhead for a tiny, one-off verification but pays off as soon as protocols, fraud patterns, or verification volume grow. This was judged worthwhile because the project's goal is a reusable framework, not a single one-time check.

6.3 Support for Future Enhancements

- New fraud patterns: added to the Fraud Pattern Library without changing the parser, constructor, or checker.

- New protocol formats: isolated to the Protocol Spec Parser, since every later stage only depends on the internal automaton representation, not the original input format.
- Integration with external systems: the CI/CD pipeline can already trigger re-verification automatically whenever a protocol specification changes, and the same REST API Gateway could later expose the framework to other tools.
- Long-term maintainability: because each verification stage is a separate component behind a stable API, individual stages (e.g. the model-checking algorithm) can be optimized or replaced without touching the presentation layer or the data layer.

Rubric — Marks Awarded

Criteria	Weight	Marks
System Requirement Analysis	30%	/5
Selection of Software Architecture	25%	/5
Architecture Diagram and Component Design	20%	/5
Component Interaction and Quality Attribute Analysis	15%	/5
Architectural Justification and Technical Presentation	10%	/5
TOTAL	**100%**	**/25**